

AIBuildAI LLM-Post-Train Agent: an agent for autonomous post-training of large language models

AIBuildAI Team

Abstract

Post-training decides whether a pretrained language model is useful in practice, but building the pipeline that carries it out is costly and demands expertise. We present an agent that does it autonomously, measured on PostTrainBench. Its core is a meta search: instead of executing one fixed search, a meta agent investigates each task and writes the search program that the run then executes, drawing on a knowledge system that covers post-training workflow, methods, datasets, and frameworks. Every result in this report was produced by meta search. Across the benchmark's seven tasks and four base models, it reaches the highest weighted average of any agent on the leaderboard, leads all agents on three tasks, and is within two points of the leading agent on each of the other four.

Correspondence: pengtao.xie@aibuildai.io

AIBuildAI ■

1 Introduction

Post-training turns a pretrained language model into one that follows instructions, reasons about its own outputs, and refuses harmful requests, and it is the stage that decides whether a model is useful in practice. The dominant recipe is well understood in outline — supervised fine-tuning followed by preference alignment — but building the pipeline that carries it out is expensive and demands expertise, because the decisions interact: the right preference algorithm depends on the data mix, the data mix that works depends on the base model, and the learning rate that converges for one algorithm diverges for another. PostTrainBench formalises this problem as a benchmark: given a pretrained base model and a target capability, an agent must choose and execute a post-training pipeline whose result is scored on the benchmark's own held-out evaluation [1]. Thirteen agent entries have been ranked on it under its current rules; the strongest is Locus, an external submission built on Opus 5, followed by the Claude Code harness running Fable 5.

Applying a single coding agent to this setting works, but it commits to one approach early and follows wherever its trajectory leads. Running several in parallel changes little, because nothing makes their directions differ and nothing moves effort off a line of attack that has stalled. Tree search over whole experiments is a better starting point, and earlier AIBuildAI systems have shown it to be effective on Kaggle-style model development and on scientific modelling tasks [2, 3]. It is not, on its own, sufficient here. A fixed tree topology forces every task through the same structure, and some of what post-training asks for it cannot express at all: a run that has to correct itself part-way through, or one whose answer is a combination of several runs rather than any single one. Which topology a post-training task needs is itself something the agent has to discover.

This report describes the system that produced our results (Fig. 1). Its core is a meta search (Section 3.2): instead of executing a fixed search, a meta agent investigates the task, consults the knowledge system, and writes the search program that the run will then execute, so that the topology is chosen for the task rather than fixed in advance. Every result reported here was produced this way. The meta agent, and every agent its program spawns, draws on a knowledge system (Section 3.1): four sub-corpora — workflow, methodology, dataset, and framework — served through a remote retrieval service that any agent queries when it faces a design decision, so that a pipeline is chosen from recorded evidence about post-training methods, datasets, and libraries rather than from the model's parametric knowledge alone. The

whole-experiment tree search of the earlier systems cited above is summarised in Section 2.2 as background: it is the fixed topology that meta search generalises, not a contribution of this report.

Section 4 reports AIBuildAI’s current results on the PostTrainBench leaderboard. On the leaderboard’s weighted average over the seven tasks and four base models, the system scores 46.6, the highest of any agent, against 45.6 for Locus and 41.8 for Fable 5. It produces the highest agent score on three tasks — AIME 2025, BFCL, and HumanEval, the last on every one of the four base models — and on each of the other four it is within two points of the leading agent. The official instruct models released by the base models’ developers remain ahead, at 51.1 on the weighted average and on every task except BFCL. These numbers are a snapshot of a system still under construction rather than a final standing on the benchmark.

2 Preliminaries

2.1 Problem formulation

The task is the one PostTrainBench defines [1]. The input is a pretrained base model together with a target capability and a fixed compute budget; the output is a post-trained checkpoint, which the benchmark scores on a held-out evaluation the agent never sees. Everything between the two is the agent’s to decide, and it amounts to a full post-training pipeline: which alignment algorithm to use, which data to train on and in what mixture, what learning rate schedule and KL penalty to set, how many epochs to run, and whether to stack a supervised fine-tuning stage before a preference stage. A task might ask for competition mathematics from gemma-3-4b-pt, scored on AIME 2025, or for function calling from Qwen3-1.7B-Base, scored on BFCL. Neither is settled by knowing the target capability: in the first the binding obstacle may be that the base model cannot emit an answer in the integer format the evaluator parses, and in the second that the emitted call does not validate against the schema, so a pipeline aimed squarely at mathematical reasoning or at tool use can score zero for reasons that have little to do with either. Compounding this, every attempt is expensive — obtaining a score means training a model through one or more post-training stages, hours of wall clock on a scarce accelerator — so only tens of pipelines fit inside a realistic budget, and one that turns out to have been misconceived costs as much as one that works. Most entries on the PostTrainBench leaderboard solve the task by pointing a general coding agent at it: Claude Code or Codex, for example, is given the base model, the data, and the budget and left to run unattended [4, 5], and this works well enough to produce most of the published results on the benchmark. What a single long-context agent does not do, however, is explore different directions: it commits to one account of the task early, every later decision is conditioned on the trajectory it has already written, and a line of attack that stalls tends to be refined rather than abandoned for one of a different kind.

2.2 Whole-experiment tree search

A possible initial attempt at this task, and at the limitation just described, is to apply the search algorithm of AIBuildAI Science and AIBuildAI-2.5, developed and evaluated there on Kaggle-style model development and on scientific modelling tasks [2, 3]. Specifically, a run is organised as a tree $\mathcal{T} = (V, E)$ whose nodes $v \in V$ are whole experiments — one pipeline built, trained, and scored — each holding the checkpoint it produced and the reward m_v that checkpoint earns under the task’s own evaluation, and whose edges each carry a textual revision instruction naming a concrete change to the parent pipeline. The value of a run is $\max_{v \in V} m_v$, the best reward any single experiment achieved.

The defining choice is that one agent, the *experimenter*, conducts each experiment end to end within a single session: it settles the approach, writes and debugs the code, runs the training and watches what it does, intervening when the loss plateaus or the reward signal collapses rather than waiting for a doomed run to finish, and reads back what the outcome implies for what to try next. The diagnosis that drives the next experiment therefore rests on first-hand execution evidence rather than on a summary handed between specialists. Around that session sit two supporting roles. A *designer* seeds the run with n_d deliberately dissimilar starting plans, each the root of a lineage that develops in its own context, so the run begins by testing several accounts of what the task needs instead of several settings of one. A *selector* orders the pool of proposals waiting for hardware — a completed experiment returns up to n_r of them, each becoming a child experiment — weighing expected gain over the parent, grounding in measured results, novelty against the rest of the tree, and cost against free capacity, and deferring rather than discarding what it ranks last. Once a session closes, the task’s

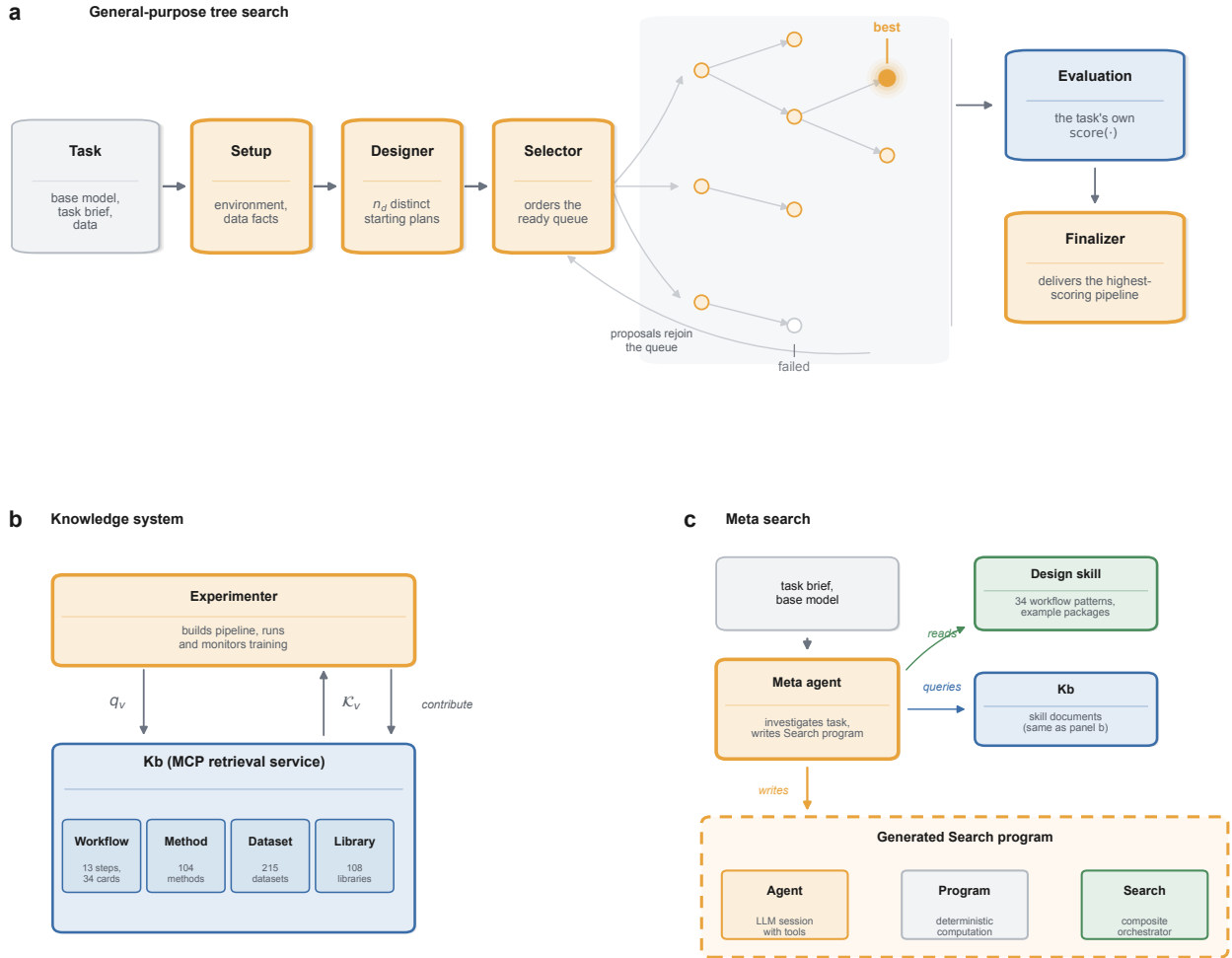


Figure 1 | System overview. **a**, Whole-experiment tree search, inherited from earlier AIBuildAI systems [2, 3]. A setup agent prepares the environment; a designer proposes n_d distinct starting plans; a selector orders the experiments waiting to run; and each finished experiment returns proposals that rejoin the queue. Every finished experiment is scored by the task’s own evaluation program, and a finalizer delivers the highest-scoring one. **b**, Knowledge system. The experimenter queries a remote retrieval service (the Kb) with a problem description q_v and receives the matching skill documents $\mathcal{K}_v$. The Kb serves four sub-corpora: a workflow skill (13 steps, 34 cards), 104 method cards, 215 dataset cards, and 108 library cards. Finished experiments may contribute observations back. **c**, Meta search, which produced every result in this report. A meta agent investigates the task, reads the design skill (34 workflow patterns with example packages) and the Kb, then writes a search program composed from three primitives: Agent, Program, and Search.

own evaluation program scores the artefacts it left behind and returns the reward m_v , so rewards are comparable across the whole tree and no agent can see the number from inside the session that earned it.

3 Method

Applied to post-training, the search of Section 2.2 exposes two limitations, and the two components described below address them in turn. The first is that it carries no knowledge of the domain it is now working in. Its agents were equipped for Kaggle-style modelling and for scientific tasks, so an experimenter facing a post-training decision must answer from parametric knowledge alone, and it answers unevenly: methods no available library ships, datasets whose contents overlap the evaluation the run is scored on, hyperparameters tuned for a different model scale. Each costs a full training run to discover. Section 3.1 describes the corpus built to close that gap, covering the workflow of a post-training run together with the methods, datasets, and frameworks it draws on. The second limitation is that the search commits to one topology

before the run begins — every run grows a tree whose nodes are whole pipelines and whose edges are revisions of them — and post-training tasks are not all of that shape. Which structure a task needs depends on where its bottleneck sits, and that is itself something the agent has to discover by working on the task rather than something the harness can fix in advance. Section 3.2 describes the meta search, which makes the topology the output of an agent and which produced every result in Section 4.

3.1 Knowledge system

Underlying the agents is a remote retrieval service, the Kb, which serves a corpus $\mathcal{K}$ of skill documents through an MCP interface. The corpus is organised into four sub-corpora, each covering one layer of the post-training problem (Appendix A gives a concrete excerpt from each):

1. **Workflow** (1 skill, 34 reference cards). The research flow of one post-training run: thirteen ordered research actions, each naming the judgment it settles, walking one concrete case, listing the directions open at that point, and marking where the case does not transfer. It owns the order in which decisions are made and the budget between them; dataset choice, method choice, and trainer choice are delegated to the three sub-corpora below.
2. **Methodology** (104 method cards, 30 with a shipped implementation). Each card carries the method’s defining paper and its math, a worked numerical example, the knobs and their defaults per library, the cost in theory and in practice, and the training signals to watch while a run is live. The 74 methods with no implementation note what building the trainer would take. A recommended table lists the 30 implemented methods with a one-sentence when-to-pick verdict; the full index groups all 104 with the variant-of chain and the data shape each one needs.
3. **Dataset** (215 dataset cards). Each card carries the licence, the exact columns and splits, a revision-pinned load line, a real sample row, sourced adoption evidence, and a screening record. The recommended set names 13 datasets covering the common training shapes; the full index groups all 215 by kind and marks the 34 that are evaluation benchmarks and must be held out.
4. **Framework** (108 library cards). Each card covers when to pick a library, how to start the run, what to watch while it runs, and how to save a result that something else can load. A recommended table lists 16 libraries with the methods each one ships and whether it is still actively maintained; the full index carries all 108 with the same measured facts.

Each skill $k \in \mathcal{K}$ is a structured record rather than a tutorial: it names approaches that succeed, the conditions under which they do, and the quantitative outcomes they produce, together with approaches that fail, their failure modes, and the conditions that trigger them, so that both positive and negative evidence are available to the agent that reads it.

When the experimenter at node v faces a design decision, it forms a query q_v from the context it holds — the task brief, the current plan, and the problem it has encountered — and retrieves the subset $\mathcal{K}_v \subset \mathcal{K}$ whose document embeddings $\phi(k)$ are nearest to $\phi(q_v)$. Because retrieval operates over the full text of each skill, a query phrased as a problem description matches skills that discuss the same problem in different terms, so the agent is not limited to techniques it can name in advance. The experimenter then conditions its design on $\mathcal{K}_v$ rather than on the model’s parametric knowledge alone: it reads what worked, what failed, and under what conditions, and builds its approach from that material. The corpus is built offline by an incremental pipeline that embeds only new or changed documents, and it evolves across runs: each finished experiment may contribute observations back to the corpus, so that later experiments inherit what earlier ones learned.

3.2 Meta search

The tree search of Section 2.2 is limited in the search strategies it can express, and meta search is meant to remove that limitation. A sweep before training begins, a tournament after it ends, a fan-in that averages several models: none is a tree of revisions, so none can be run. Meta search removes the restriction by making the search topology itself the output of an agent. Rather than executing a fixed search, the system first spawns a *meta agent* that investigates the task, consults the knowledge system, and writes a *search program*: a structured Python package that defines the search the run will then execute (Appendix B reproduces one, generated for AIME 2025 with Qwen3-1.7B-Base). The search program is composed from three primitives. An *Agent* is an LLM session with tools, policies, and a typed output; a *Program* is a deterministic computation that runs in a managed worker with its own resource limits; and a *Search* is a composite that orchestrates Agents, Programs, and nested Searches through spawn, run, and wait operations, with typed inputs and

Table 1 | Current results on PostTrainBench. Rows are methods; columns are the benchmark’s seven tasks and the overall score. Each task entry is the score the post-trained model achieves on the held-out evaluation, averaged over the four base models (Qwen3-1.7B-Base, Qwen3-4B-Base, SmolLM3-3B-Base, gemma-3-4b-pt) and given as mean $\pm$ s.d. across them. *Overall* is the leaderboard’s weighted average over the seven tasks (AIME 22.7%, GPQA 22.5%, HealthBench 18.4%, HumanEval 10.6%, GSM8K 9.4%, ArenaHard 9.0%, BFCL 7.5%), with the s.d. across seeds that the leaderboard reports. Locus, an external submission by Intology on Opus 5, and Claude Code running Fable 5 are the two strongest entries on the leaderboard; Claude Code running Opus 5 uses the model on which every AIBuildAI agent runs. Fable 5’s GPQA entry is the leaderboard’s Opus 4.8 (Max) score, substituted after Fable 5 refused that benchmark. *Official instruct* is the instruction-tuned release of each base model by its developers, a reference outside the ten-hour budget rather than an agent. Bold marks the best agent per column; Appendix C lists every leaderboard entry.

Method	AIME	ArenaHard	BFCL	GPQA	GSM8K	HealthBench	HumanEval	Overall
AIBuildAI (ours)	15.8 $\pm$ 11.0	65.5 $\pm$ 18.6	95.8 $\pm$ 1.7	31.4 $\pm$ 2.7	82.3 $\pm$ 8.4	42.6 $\pm$ 4.3	69.0 $\pm$ 11.1	46.6
Locus	9.4 $\pm$ 9.2	66.3 $\pm$ 16.8	94.3 $\pm$ 1.2	33.1 $\pm$ 3.6	82.9 $\pm$ 5.5	44.3 $\pm$ 3.9	66.6 $\pm$ 12.8	45.6 $\pm$ 0.8
Claude Code (Fable 5)	13.3 $\pm$ 10.8	61.5 $\pm$ 18.4	73.0 $\pm$ 28.0	28.3 $\pm$ 2.3	83.5 $\pm$ 6.1	40.1 $\pm$ 5.7	58.4 $\pm$ 19.3	41.8 $\pm$ 1.7
Claude Code (Opus 5)	9.2 $\pm$ 9.2	53.4 $\pm$ 22.3	1.5 $\pm$ 3.0	31.4 $\pm$ 4.1	80.3 $\pm$ 11.5	38.7 $\pm$ 6.9	59.8 $\pm$ 21.1	35.0 $\pm$ 1.4
<i>Official instruct</i>	29.2 $\pm$ 17.9	70.2 $\pm$ 24.0	85.0 $\pm$ 13.0	36.2 $\pm$ 5.9	87.0 $\pm$ 5.3	43.3 $\pm$ 9.8	71.5 $\pm$ 4.0	51.1

outputs flowing between them. In the formalism of Section 2.2, a fixed search defines one family of trees $\mathcal{T}$; meta search defines a mapping $\pi : \mathcal{I} \rightarrow \mathcal{F}$ from the task information $\mathcal{I}$ available to the meta agent to the family of execution graphs $\mathcal{F}$ the run will explore, where $\mathcal{F}$ is strictly larger than the set of trees, since it includes DAGs, tournaments, fan-ins, and iterative loops. The value of the run remains $\max_v m_v$ over all scored nodes, but the topology that produced those nodes is now task-specific.

Two resources support the meta agent in writing the search program. The first is a design skill containing 34 workflow and search patterns, each mapped to the execution SDK with a complete, loader-verified example package. Many are general — single-shot, sequential chain, router–specialists, DAG with joins, map–reduce, parallel best-of- n , tournament, committee voting, orchestrator–workers, evaluator–optimizer, and others — and several are specific to post-training, among them supervised fine-tuning followed by GRPO, teacher distillation with iterative refinement, a checkpoint tournament with weight averaging, and runtime replanning. The patterns are a vocabulary, not a whitelist: the meta agent may combine, adapt, or invent new structures when the evidence it gathers from the task points a different way. The second is the knowledge system of Section 3.1, which the meta agent queries the same way an experimenter does, retrieving skill documents whose recorded successes and failures inform its choice of topology.

We give three post-training cases in which the unit the task asks us to compare is not a unit a tree node can be. (1) Generating the training data. The unit is a corpus — sampled and filtered completions, or preference pairs built by sampling several responses and ranking them — but a tree node has to be a scored pipeline, and a corpus scores only once a model has been trained on it, so trying k recipes costs k training runs. A program makes generation a stage in its own right and screens corpora on cheap checks such as verifier pass rate and duplication. (2) A multi-stage curriculum — supervised fine-tuning, then preference optimisation, then rejection fine-tuning — whose ordering is unknown. The unit is a stage; a tree node is a whole pipeline, so every candidate sharing a prefix retrains it from scratch, where a program caches the prefix and searches only over what follows. (3) Online reinforcement learning against a learned reward model, where the policy may begin exploiting the reward part-way through a run. The unit is a segment of one continuous run — policy steps, a refit of the reward model on fresh preference data, then a resume from parameters that have already moved — whereas a tree node is trained from its parent’s configuration and scored only once it has finished, so the search can only discard the run and start another.

A search program also need not fix the whole run in advance. Under the runtime-replanning pattern, the program a meta agent writes carries out one stage — building a corpus, one training round with its evaluation, or one continuation of an earlier model — then reads the remaining budget and hands the run to a new meta agent, which reads what the stage produced and writes the program for the next stage. The structure of the run is then decided one stage at a time, on measured outcomes rather than on a forecast made before any training; Section 4.3 traces two runs of this kind.

4 Current results

Our results for AIBuildAI on the PostTrainBench leaderboard are given below.

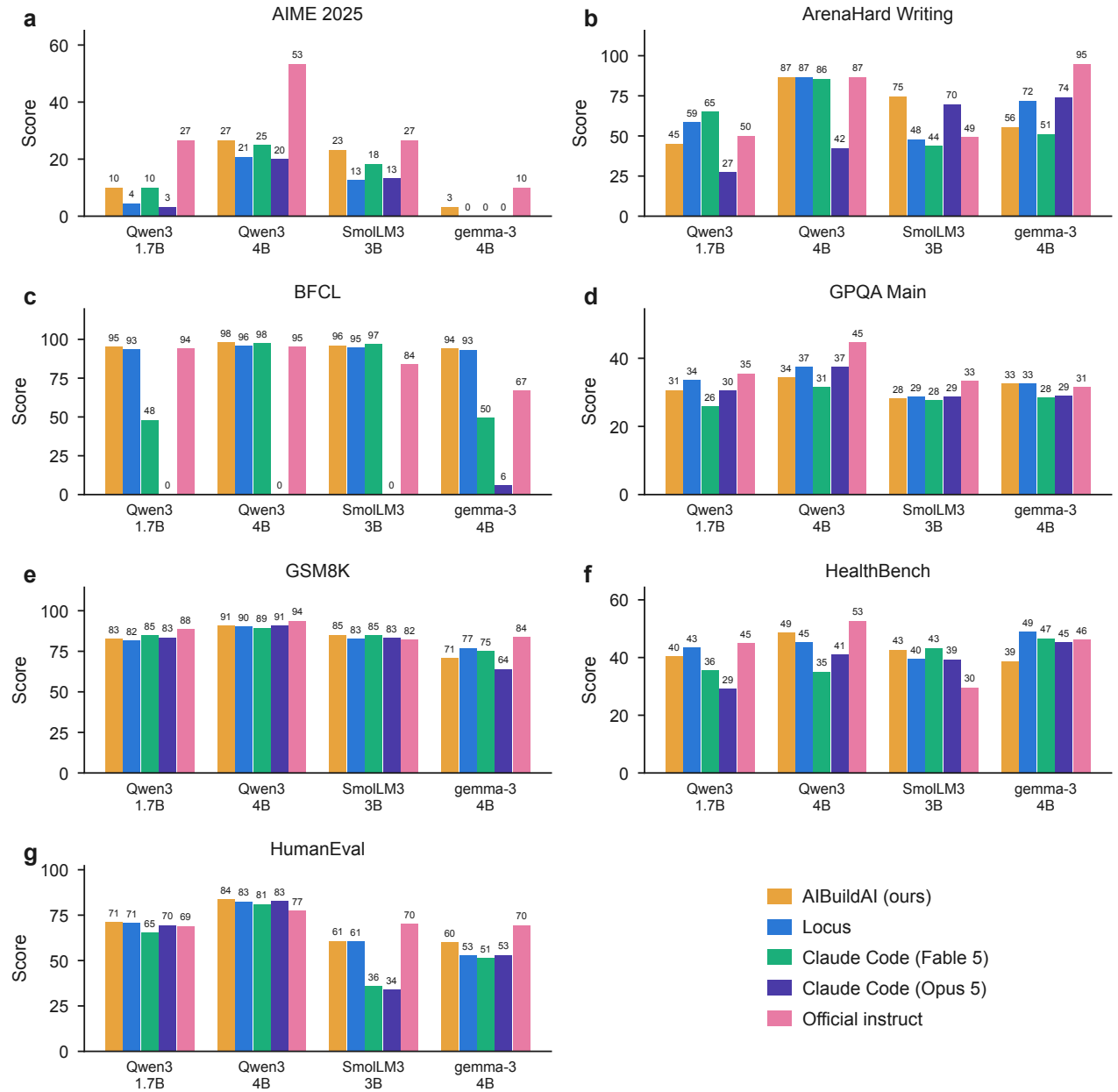


Figure 2 | Scores on each PostTrainBench task, per base model. a–g. One panel per task, with one group of bars per base model and one bar per method. Each bar is the score the post-trained model achieves on that task’s held-out evaluation; bar labels are rounded to the nearest integer. *Official instruct* is the instruction-tuned release of the same base model by its developers and is a reference rather than a competing agent. The vertical scale differs between panels, since the tasks are scored on different quantities. These are the per-model values that Table 1 averages.

4.1 Benchmark and experimental setup

In PostTrainBench, each task supplies a pretrained base model and a target capability; the agent must choose and execute a post-training pipeline whose result is scored on the benchmark’s own held-out evaluation [1]. We evaluate on four base models — Qwen3-1.7B-Base, Qwen3-4B-Base, SmolLM3-3B-Base, and gemma-3-4b-pt — across the benchmark’s seven tasks, which span mathematical reasoning (AIME 2025, GSM8K), creative writing (ArenaHard Writing), general question answering (GPQA Main), health advice (HealthBench), code generation (HumanEval), and function calling (BFCL). Each task ran fully autonomously under a ten-hour wall-clock budget on a single H100 GPU, the budget and hardware the leaderboard sets. Every result was produced by meta search (Section 3.2), with every agent role running on

Claude Opus 5. No human intervened in any run. Baseline numbers are the published values from version 1.1 of the PostTrainBench leaderboard, which ranks thirteen agent entries. The main comparison (Table 1) uses three of them: Locus, an external submission run by Intology on Opus 5 and first on the leaderboard; the Claude Code harness running Fable 5, second; and Claude Code running Opus 5, the model every AIBuildAI agent runs on. We shorten the last two to Fable 5 and Opus 5 below. Appendix C lists all thirteen, together with the two references the leaderboard reports for each combination: the official instruct model and the untouched base model.

4.2 Where the system stands

Table 1 reports all seven tasks, each averaged over the four base models, and Fig. 2 breaks each of them out by base model. On the leaderboard’s weighted average the system scores 46.6, one point above Locus at 45.6 and 4.8 above Fable 5 at 41.8. It produces the highest agent score on three tasks, AIME 2025, BFCL, and HumanEval; Locus leads on ArenaHard Writing, GPQA Main, and HealthBench, and Fable 5 on GSM8K, and on each of those four the system is within two points of the leader. We take the tasks in turn.

The three leads differ in what they reward. BFCL scores a function call by whether it parses and matches the expected form, so the task is gated by a single output contract: a pipeline that teaches the contract scores highly and one that does not scores near zero, as Opus 5’s 1.5 against the system’s 95.8 shows. It is the one task on which the system also exceeds the official instruct models, at 95.8 against 85.0. HumanEval rewards code that passes unit tests; the system produces the highest agent score on every one of the four base models, for an average of 69.0 against Locus’s 66.6. AIME 2025 carries the largest weight in the overall score, 22.7%, and on it every agent remains far below the official instruct models, which average 29.2. The system’s 15.8, against 13.3 for Fable 5 and 9.4 for Locus, comes from the highest or joint-highest agent score on every base model, including gemma-3-4b-pt, where it reaches 3.3 and no other agent exceeds 1.1.

ArenaHard Writing and HealthBench are both graded by an LLM judge — a pairwise preference judgement of writing on the first, a physician-written rubric of weighted criteria for medical advice on the second — and the untouched base models average only 1.3 and 9.5 on them, so almost the whole score is won in post-training. The system trails Locus by 0.8 on ArenaHard Writing and by 1.7 on HealthBench, and on one base model of each it produces the highest agent score: 74.6 on ArenaHard Writing with SmoLM3-3B-Base, where Locus reaches 48.0 and the official instruct model 49.2, and 48.6 on HealthBench with Qwen3-4B-Base, against Locus’s 45.3. Section 4.3 traces both runs. The remaining gap is concentrated on a few combinations: on ArenaHard Writing the system trails Locus by 13.4 points with Qwen3-1.7B-Base and 16.3 with gemma-3-4b-pt, and on HealthBench by 10.2 with gemma-3-4b-pt.

GPQA Main and GSM8K are the closest of the four. On GPQA Main the system’s 31.4 is 1.6 below Locus’s 33.1, against per-model spreads of about 3 for both, and with gemma-3-4b-pt it produces the highest agent score, 32.6, above the official instruct model’s 31.5. On GSM8K the three strongest entries lie within 1.3 points of one another — Fable 5 at 83.5, Locus at 82.9, the system at 82.3 — and with SmoLM3-3B-Base the system produces the highest agent score, 85.1, again above the official instruct model’s 82.2. What separates them is gemma-3-4b-pt, on which the system reaches 70.7 against Locus’s 76.9.

4.3 Two runs

Two runs show what a search written for its task, one stage at a time, does with a ten-hour budget. Both followed the runtime-replanning pattern of Section 3.2, and in both the search program served an open-weight teacher model on the run’s own GPU to write training data. (1) On HealthBench with Qwen3-4B-Base, whose untouched score is 13.4, the run passed through four meta agent generations. The first built a corpus of 47,744 teacher-written responses. The second trained on it for 1,300 steps and confirmed a checkpoint at 46.2 on the full evaluation set. The third had the teacher write 6,751 longer multi-turn conversations, continued training on them for 650 steps, screened six candidates on a 32-item subset against a gain threshold it had fixed before measuring, and confirmed the winner at 48.6 on the full set; the fourth returned that model. (2) On ArenaHard Writing with SmoLM3-3B-Base, whose untouched score is 0.4, the first generation served the teacher to build a corpus of 20,516 rows and ran a pilot fine-tune on part of it. The second retrained on the full corpus for 2,600 steps and averaged the weights of three of its last checkpoints, reaching 74.2 on the full set. The third ran a rejection-sampling round: it trained for a further 800 steps on 5,805 rows — 2,246 of the student’s own responses that the teacher ranked best, 76 written by the teacher, and 3,483 replayed rows — and again averaged three checkpoints, reaching 74.6, the model the run delivered. In both runs almost the whole gain came from

the first full training stage on teacher-written data, and each later stage kept its result only after confirming it on the full evaluation set rather than on the subset screens.

5 Next steps

The work ahead runs in two coupled directions. The first is to extend the knowledge system so that it carries search structure and not only training content. The corpus as described in Section 3.1 records what to train — which method, on which data, with which trainer — but says nothing about how to organise the search that finds it. For a task of a recognisable kind, the corpus should be able to state which search scheme has worked before, under what conditions, and what it returned, so that the meta agent begins from a structure with evidence behind it instead of composing one from nothing. A prior of that kind changes the meta agent’s job from open-ended design to selection and adaptation, which is both a cheaper problem and a more reliable one: two runs on the same kind of task should not arrive at unrelated topologies for reasons neither can account for.

The second concerns the space the meta agent designs in. The primitives of Section 3.2 compose almost without restriction, and while that expressiveness is what allows a topology to fit a task, it also means the agent selects from a space far larger than the set of schemes that are actually useful for post-training. A larger space is not a free option: it raises the uncertainty of any single design and the variance between runs, and it does so without a corresponding gain, since most of what it admits no task would want. What we are after is a space that is compact enough to make the agent’s choice well posed and expressive enough to represent the schemes that matter — sweeps, tournaments, fan-ins, staged pipelines, and their combinations — for post-training and, we expect, for AI development tasks more generally. Finding that space, rather than widening the vocabulary further, is where the meta search goes next.

Data Availability

The PostTrainBench benchmark data used in this study are publicly available from the PostTrainBench repository [1]. The leaderboard values used as baselines are published on the PostTrainBench leaderboard. The per-task scores and submitted code for every run reported here accompany this report.

Code Availability

The AIBuildAI software is publicly available at <https://github.com/aibuildai/AI-Build-AI>, together with instructions for using the system.

References

- [1] Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. PostTrainBench: Can LLM agents automate LLM post-training?, 2026. URL <https://arxiv.org/abs/2603.08640>.
- [2] Peijia Qin, Ruiyi Zhang, Qi Cao, Han Guo, Li Zhang, and Pengtao Xie. AIBuildAI-2.5: Efficient autonomous AI model development through LLM-guided tree search. Manuscript, <https://www.aibuildai.io/#guide-max>, 2026. Accessed: 2026-08-24.
- [3] AIBuildAI Team. AIBuildAI Science: Autonomous development of AI models for scientific research. Manuscript, <https://www.aibuildai.io/#guide-science>, 2026. Accessed: 2026-08-24.
- [4] Anthropic. Claude code. <https://code.claude.com/docs/en/overview>, 2026. Accessed: 2026-08-22.
- [5] OpenAI. Codex CLI. <https://github.com/openai/codex>, 2026. Accessed: 2026-08-22.

A Skill document examples

The knowledge system described in Section 3.1 is organised into four sub-corpora. Below we reproduce one representative excerpt from each, lightly abridged to fit. Every claim on a card carries a URL and a commit or fetch date; nothing comes from the system’s own runs.

A.1 Workflow: understanding what the scoring rewards

The workflow skill opens with the instruction to work through every step in order. Its first step reads:

Skill: Post-training workflow — Step 1.

Judgment Understand what the scoring rewards — and price it.

“This is a code task” or “this is a math task” is still too coarse. Settle these before any data is chosen: does the score read the final answer or the whole reply; does a format error score zero on its own; does reasoning earn anything or only add truncation risk; can an answer be checked automatically by execution, by an answer key, or by a schema; is there one score or several dimensions; and do inference-time sampling settings affect the official number. Then price it. A number you did not produce yourself is not a baseline. Run the shipped evaluator once on the untouched base model, on the full set or on a fixed slice if the full set is too slow. The number it returns is the baseline, and it prices the floor under every later decision.

Case A function-calling task scores 0.04 after SFT with the shipped sampling defaults, then wins nearly the whole task back without touching weights, because the failure lives in the evaluator’s own generation path rather than in the model. The same baseline measurement also gives the per-evaluation price the budget uses later.

Directions Read the shipped evaluation code, not any prose description of the task. Write down the metric, the extraction rule, the token cap, and any retry ladder. Time the baseline run, so the price of one full evaluation is a measured number.

A.2 Methodology: DPO

The DPO card opens with a one-sentence summary and a when-to-pick verdict, then gives the method’s math:

Skill: DPO (Direct Preference Optimization).

When to pick Pick DPO when you already have (or can cheaply construct) a static dataset of paired preferences and do not want to run generation inside the training loop. It is the offline alternative to PPO-based RLHF.

Objective $\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$

Worked example $\beta = 0.1$. For one pair: $\log \pi_{\theta}(y_w|x) - \log \pi_{\text{ref}}(y_w|x) = 0.5$, $\log \pi_{\theta}(y_l|x) - \log \pi_{\text{ref}}(y_l|x) = -0.3$. Loss argument: $\beta(0.5 - (-0.3)) = 0.08$, so $\mathcal{L} = -\log \sigma(0.08) \approx 0.653$, close to the 0.693 a completely unconfident model would produce.

Data A static dataset of (prompt, chosen, rejected) triples. Offline: the same fixed dataset is reused across the whole run, with no fresh sampling from the policy during training.

Extra models One frozen reference model (forward passes only). No value network, no explicit reward model.

Shipped by trl (DPOTrainer), with loss variants: sigmoid (default), IPO, hinge, SimPO-normalised.

A.3 Dataset: HuggingFaceH4/ultrafeedback_binarized

Each dataset card opens with a use-it-for verdict and a restriction, then gives the exact load line:

Skill: ultrafeedback_binarized.**Shape** preference pairs**Rows** 187,405**Licence** MIT**Restriction** Hold out all three test splits, and read the card: messages means a different thing in each split.**Load it**

```
from datasets import load_dataset
ds = load_dataset(
    "HuggingFaceH4/ultrafeedback_binarized",
    revision="53a10...",
    split="train_prefs",
)
```

Sample row A prompt asking to explain quantum computing, with a chosen completion rated 5/5 and a rejected completion rated 2/5, each carrying the source model that generated it.**Screening** Licence check, column audit, and overlap test against known benchmarks are recorded in the card's appendix.

A.4 Framework: trl

The trl card opens with a one-sentence verdict and a quick start:

Skill: trl (Transformers Reinforcement Learning).**When to pick** Post-training in the Hugging Face ecosystem, when your models and datasets live on the Hub and you want a trainer-class API that starts on one machine with a wide method menu (16 trainers) and scales out through Accelerate.**Methods** Online: GRPO, RLOO, Online DPO, Nash-MD, PPO, XPO. Reward modelling: Reward, PRM. Offline: SFT, DPO, KTO, BCO, CPO, ORPO. Distillation: GKD, MiniLLM.**Quick start**

```
from trl import SFTTrainer
from datasets import load_dataset
trainer = SFTTrainer(
    model="Qwen/Qwen2.5-0.5B",
    train_dataset=load_dataset(
        "trl-lib/Capybara", split="train"),
)
trainer.train()
```

Scale Single GPU to multi-node via Accelerate and DeepSpeed. Sequence-parallel options (Ring Attention, Ulysses) for long-context training.**Install** `pip install trl; v1.9.2; Apache-2.0.`

B A generated search program

The meta search described in Section 3.2 writes a complete search program before the run begins. Below we reproduce one such program, generated for AIME 2025 with Qwen3-1.7B-Base, lightly abridged for clarity. The meta agent investigated the task first and recorded its findings in the program's docstring:

The untouched base model scores 1/30 on AIME 2025, emits an ANSWER: line 20% of the time and never terminates on 23% of items; giving it two format demonstrations raises the answer-line rate to 73% and moves accuracy by exactly zero while shrinking the median completion from 844 to 394 tokens. Format is therefore not the bottleneck, and the open question is which reasoning-trace length regime a 1.7B model can actually be taught.

From this evidence the meta agent designed a five-stage search. The stages run sequentially on one GPU, except that corpus construction (CPU-only) overlaps with the baseline insurance stage:

Stage A: Corpus construction and baseline insurance. Three corpora are built in parallel on the CPU, each targeting a different completion-length regime (short, mid, and long chain-of-thought). Meanwhile, on the GPU, a baseline insurance agent saves the untouched base model and runs the official evaluator once to establish the floor score and the per-evaluation cost.

```
# CPU: three corpora built in parallel
corpus_programs = [
    CorpusBuildProgram(recipe)
    for recipe in cfg.corpus_recipes # short, mid, long
]
corpus_handles = [await ctx.spawn(p) for p in corpus_programs]

# GPU: save untouched model, run evaluator once
baseline = BaselineInsuranceAgent(
    base_model=cfg.base_model,
    eval_limit=cfg.smoke_eval_limit,
    gpu_count=1,
)
baseline_handle = await ctx.spawn(baseline)

await ctx.wait(corpus_handles)
baseline_result = await baseline_handle.result()
```

Stage B: Three pilot SFT runs. One pilot per corpus, run sequentially on the single GPU. Each pilot trains on a fixed token budget (smaller than the main run) and evaluates the result against the incumbent. The incumbent starts as the untouched baseline and is replaced whenever a pilot beats it.

```
for corpus in corpora:
    pilot = PilotSftAgent(
        corpus_jsonl=corpus.jsonl_path,
        pilot_token_budget=cfg.pilot_token_budget,
        incumbent_accuracy=incumbent_accuracy,
        gpu_count=1,
    )
    outcome = await ctx.run(pilot)
    if outcome.beat_incumbent:
        incumbent = outcome
```

Stage C: Strategy selection. A strategy agent reads the baseline measurement, all three pilot outcomes, and the corpus screening reports, then decides which corpus to use for the full training run, whether to start from a pilot checkpoint or from scratch, and whether a second stage (length annealing, RFT, or DPO) is warranted.

```
strategy = await ctx.run(StrategySelectionAgent(
    baseline_summary=baseline_summary,
    pilot_summary=pilot_summary,
    corpus_summary=corpus_summary,
    gpu_count=0, # text-only reasoning
))
```

Stage D: Main training. A full SFT run on the chosen corpus with the chosen recipe, using the strategy agent’s configuration. If the strategy agent selected a pilot checkpoint, training resumes from it rather than starting from the base model.

Stage E: Optional second stage. Run only when the strategy agent requested it. The method (one of `continue_sft`, `length_anneal`, `rft`, `dpo`) is the strategy agent’s decision, conditioned on the measured pilot outcomes.

In this run, the strategy agent selected the mid-length chain-of-thought corpus with a `length_anneal` second stage, producing a final model that scored 6.67% (2/30) on AIME 2025, up from 3.33% (1/30) for the base model. The full program is 471 lines of Python across 8 files (the search orchestrator, 5 agent roles, and 2 deterministic programs), all generated by the meta agent in a single session before any training began.

C Full leaderboard

Table 2 lists every entry of version 1.1 of the PostTrainBench leaderboard, with AIBuildAI placed by its weighted average.

Table 2 | Every PostTrainBench leaderboard entry. Each agent is named as the leaderboard lists it: its harness, or for Locus its submitter, followed by its model and reasoning-effort setting. Task columns are means over the four base models; *Overall* is the leaderboard’s weighted average, with the s.d. across seeds where the leaderboard reports one. Table 5’s GPQA entry is the leaderboard’s Opus 4.8 (Max) score. *Official instruct* and *Base model* (the untouched base model, evaluated zero-shot) are references, not agents. Bold marks the best agent per column.

Method	AIME	ArenaHard	BFCL	GPQA	GSM8K	HealthBench	HumanEval	Overall
<i>Official instruct</i>	29.2	70.2	85.0	36.2	87.0	43.3	71.5	51.1
AIBuildAI (ours)	15.8	65.5	95.8	31.4	82.3	42.6	69.0	46.6
Locus (Intology, Opus 5)	9.4	66.3	94.3	33.1	82.9	44.3	66.6	45.6 ± 0.8
Claude Code (Fable 5, Max)	13.3	61.5	73.0	28.3	83.5	40.1	58.4	41.8 ± 1.7
Codex CLI (GPT 5.6 Sol, Max)	7.5	26.4	94.4	30.1	69.5	27.4	63.9	36.2 ± 0.2
Claude Code (Opus 5)	9.2	53.4	1.5	31.4	80.3	38.7	59.8	35.0 ± 1.4
Claude Code (Opus 4.8, High)	9.2	45.8	58.1	20.7	69.7	32.4	58.1	33.8 ± 3.6
Claude Code (Opus 4.8, Max)	10.8	36.1	47.1	28.3	68.9	31.7	47.3	32.9 ± 5.8
Claude Code (Kimi K3)	11.7	36.7	9.7	25.2	76.3	31.8	62.3	32.0 ± 0.3
Claude Code (GLM 5.2, Max)	7.8	27.0	47.9	29.7	72.7	26.5	52.5	31.7 ± 2.1
Claude Code (Opus 4.7, xHigh)	6.4	19.6	76.9	27.5	60.7	17.8	42.1	28.6 ± 1.7
Codex CLI (GPT 5.5, xHigh)	3.8	12.9	56.1	30.7	56.7	19.8	48.9	27.2 ± 0.6
Cursor CLI (Grok 4.5, High)	10.4	14.1	1.5	27.1	65.1	11.3	51.3	23.5 ± 0.1
OpenCode (Gemini 3.1 Pro)	3.9	7.4	63.6	18.5	45.5	16.3	40.2	22.0 ± 1.8
Codex CLI (GPT 5.4, High)	0.6	9.6	23.6	28.0	48.2	13.9	27.3	19.0 ± 3.0
<i>Base model</i>	1.7	1.3	1.5	8.5	20.4	9.5	12.8	7.5